

MAJOR PROJECT · FULL-STACK AI WEB APPLICATION

Smart Sugarcane AI

Irrigation scheduling, disease detection, soil analysis, variety ranking and fertilizer guidance — one web application, running without a single external AI API key.

smart-sugarcane-ai.vercel.app

7

AI / decision modules

43

REST API endpoints

12,000

training rows

13

React pages

React 19 · Vite 6 · TypeScript · Tailwind v4 · FastAPI · SQLAlchemy · scikit-learn · PostgreSQL

How the system fits together

A single Vercel project serves the React build and the Python API from the same origin — no CORS, no third-party AI service in the request path.



SEVEN MODULES ON THE SITE



Live vs. local — the site reports this itself at `/api/system/status`. Vercel caps a Python function at 250 MB and the scientific stack is 457 MB, so the deployed API runs the transparent water-balance rule engine instead of the trained Random Forest, and analyses uploaded photos in memory rather than storing them. Running it locally gives the trained model and stored images.

Frontend

A typed React single-page app — every screen is a route behind an auth guard, and every AI result carries a badge saying where it came from.

STACK

● React	19
● Vite	6
● TypeScript	5.7
● Tailwind CSS	v4
● React Router	7
● Axios	1.7
● Recharts	2.15
● Lucide icons	0.469

R

Routing & guards

ProtectedRoute and PublicOnlyRoute wrap the router; a public layout for the landing page, a dashboard layout with sidebar for the rest.

S

State

AuthContext holds the JWT and current user, ToastContext handles notifications, useTheme persists light / dark mode.

A

API client

One Axios instance with a bearer-token interceptor; every route typed in endpoints.ts against shared TS types.

V

Result surfaces

Recharts trend charts, drag-and-drop ImageDropzone, and a ModelBadge + HonestyNotice on every AI answer.

13 pages behind one router: Landing · Login · Register · Dashboard · Irrigation · Plant Analysis · Soil Analysis · Varieties · Fertilizer · Assistant · History · Settings · 404

Backend

FastAPI with strict one-directional layering — nothing lower ever imports from anything higher, so the training scripts and the live API share exactly one copy of the agronomy.

LAYERS

api/ routers, validation, auth dependency

services/ business logic; ML + knowledge base

ml/ rule engine, image features, loaders

models/ SQLAlchemy ORM — no business logic

43 endpoints across 10 routers

/auth 6	/irrigation 7	/plants 6
/soil 5	/recommendations 5	/assistant 5
/history 4	/weather 2	/system 2
/dashboard 1		

5 database tables

users · irrigation_records · plant_analyses · soil_analyses · chat_messages
PostgreSQL (Supabase) in production, SQLite locally — one SQLAlchemy 2.0 model layer serves both, with a custom UTCDateTime type because SQLite silently drops timezone information.

Runtime: FastAPI 0.115 · Uvicorn 0.34 · Pydantic 2.10 · SQLAlchemy 2.0 · PyJWT + bcrypt · Pillow 11 + OpenCV · NumPy · pandas · scikit-learn 1.6 · joblib · httpx — Python 3.12

Datasets used

One dataset the model is actually trained on, one hand-curated knowledge base, and the image data this project deliberately does not ship.

TRAINED ON

Irrigation dataset

12,000 rows × 17 columns (1.6 MB CSV) — 9 input features, 2 targets.
Synthetic: generated by the project's own water-balance engine and stamped dataset_type: synthetic_demonstration in the model bundle.
80 / 20 split → 2,400 held-out test rows.

CURATED

Agronomy knowledge base

4 editable JSON files, ~60 KB, each versioned and dated:
12 sugarcane varieties (Co 86032, Co 0238, CoC 671, CoM 0265 ...),
7 disease classes + unknown, 6 soil profiles, 5 growth-stage rule sets.
Hot-reloadable through the API — no code change to correct advice.

NOT BUNDLED

Image datasets

No labelled sugarcane photo set ships with the project, so disease and soil run as clearly-labelled DEMO heuristics rather than a model trained on nothing. Documented training path: one folder per class, 100 images minimum / 500+ ideal — Kaggle, Mendeley, Roboflow, PlantVillage.

GENERATED AT RUNTIME

Your own records

Every analysis is saved per user: irrigation_records, plant_analyses, soil_analyses and chat_messages.
These feed the dashboard trend chart, the plant-health history and the assistant, which answers only from your own saved records.

The 9 irrigation features: soil moisture · temperature · humidity · rainfall · rain probability · wind speed · weather condition · soil type · growth stage

Targets: water_requirement_mm (regression) and irrigation_required (classification). Swap in a CSV of your own logged field decisions and the synthetic-data warnings disappear from the app.

Algorithms behind each answer

Six engines, each chosen for the data actually available to it — and each one declares its own type in the response.

1 Irrigation

RandomForestRegressor (water in mm) plus a RandomForestClassifier (irrigate or not) over the 9 features, wrapped around an FAO-56-style water balance: $ET_o \times Kc - \text{effective rain}$. If the classifier disagrees with the balance, the app says so.

2 Water balance

The transparent fallback, and the source of the training labels. Kc 0.50 → 1.25 by growth stage, moisture triggers per soil type, application efficiency 0.55 flood → 0.90 drip, rainfall effectiveness 0.75. Every constant is documented.

3 Disease · DEMO

HSV colour masks (green / yellow / brown / dark / bleached), brown fragmentation, edge density and streak orientation, scored per class. Fragmentation is what separates rust from red rot. Confidence is capped at 0.88; below 0.42 it answers "unknown".

4 Soil · DEMO

A weighted mean of membership functions over hue, saturation, brightness and graininess → 5 soil classes. A mean, not a sum, so one failed criterion drags the score down. It reports appearance only — a photo cannot measure NPK, pH or conductivity.

5 Variety ranking

Weighted multi-criteria fit across 12 varieties: soil 30 % · region 24 % · water 20 % · climate 12 % · season 9 % · irrigation 5 %. Returns a ranked list with the reasoning and the cautions attached to each variety.

6 Fertilizer & assistant

Fertilizer: stage rule engine, 5 growth stages × 6 soil types → nutrient priorities and a split schedule, with no kg/ha dosages by design. Assistant: keyword intent matching over your saved records and the JSON base, citing which records it used. Not an LLM.



Trained model (local run): R^2 0.962 · MAE 1.35 mm · accuracy 96.4 % · F1 0.94 on 2,400 held-out rows.

Those numbers measure how faithfully the forest reproduces the simulation it was trained on — not field accuracy. Every API response carries `model_source` (`trained_model` / `rule_engine` / `demo_heuristic`) and the UI renders a badge for it, so nothing is presented as more than it is.